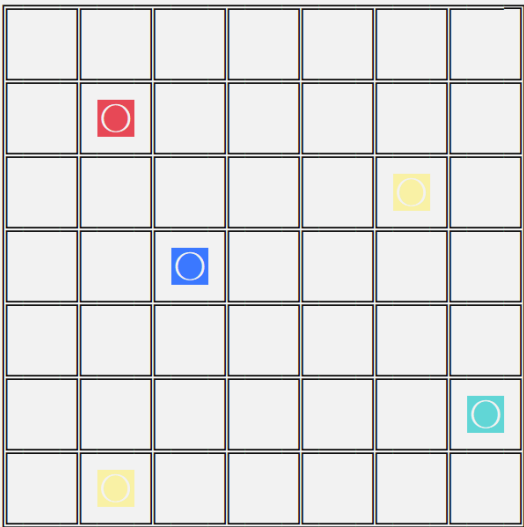


鼠标移动，左键单击选择，右键退出



[当前光标] 位置非法

得分：0



	:44/ (89.80%)	消除-0
	:00/ (0.00%)	消除-0
	:01/ (2.04%)	消除-0
	:00/ (0.00%)	消除-0
	:01/ (2.04%)	消除-0
	:01/ (2.04%)	消除-0
	:00/ (0.00%)	消除-0
	:02/ (4.08%)	消除-0

实验报告名称：彩球游戏

班级：高程03班

学号：2452640

姓名：罗啦

完成日期：2026年6月17日星期三

1. 题目

1.1 项目背景

本次作业要求实现一个仿Windows版Colorlinez的彩球游戏程序。Colorlinez是一款经典的益智类小游戏，玩家通过移动彩球使同色球在横、竖、斜方向上连成五个或以上，从而消除得分。

1.2 彩球颜色与生成规则

游戏中共有七种不同颜色的彩球，分别用数字1至7表示。程序启动后，首先在棋盘上随机生成五个彩球，这五个球的位置必须是随机的且不能重叠，五个球的颜色也是随机从七种颜色中选取的。在后续的游戏过程中，每次移动操作后如果没有发生消除，则需要在棋盘的空位中再随机生成三个新球；如果移动后发生了消除，则不生成新球，相当于给玩家一次“免费”的消除机会。每次生成新球时，三个球的颜色也是随机确定的。

1.3 基本操作与移动规则

玩家通过鼠标与游戏进行交互。首先用鼠标左键单击选中某个已有的彩球，被选中的彩球需要有明显的高亮效果以区别于普通彩球。然后玩家再用鼠标左键单击一个空白的目标位置，程序需要判断从起点到目标位置是否存在一条通路。通路的定义是：从起点格子出发，可以向上、下、左、右四个方向移动，沿途只能经过空格子，不能穿越其他彩球，也不能走出棋盘边界，最终能够到达目标位置。如果存在通路，程序将彩球沿着通路移动到目标位置，并且移动过程需要有逐格移动的动画效果，动画必须能够跨越棋盘内部的横线和竖线分隔符。如果不存在通路，程序则不执行移动操作，并给出相应的错误提示。

1.4 消除规则与得分机制

彩球移动到位后，程序需要检查整个棋盘，判断是否存在五个或以上同色球在一条直线上相连的情况。检查的方向包括横向（水平方向）、纵向（垂直方向）以及两个斜向（主对角线方向和副对角线方向）。一旦检测到五个或以上同色球连续排列，这些彩球就会被消除，对应的格子变为空格。

得分计算公式为：消除数量减一乘以消除数量减二。举例说明，消除5个球时得分为4乘以3等于12分，消除6个球时得分为5乘以4等于20分，消除7个球时得分为6乘以5等于30分。对于双五连的情况，即某个彩球同时属于横向五连和纵向五连（如十字交叉的中心点），该交叉点需要重复计数，也就是说横向五连计分一次，纵向五连也计分一次，总分为两次得分之和。

1.5 新球生成与游戏结束条件

移动完成后，程序需要判断本次移动是否产生了消除。如果产生了消除，则本轮不生成新球，直接进入下一轮等待玩家操作。如果没有产生消除，则程序需要在当前棋盘的空位中随机生成三个新彩球，新球的颜色也是随机确定的。每次生成新球之前，程序应该通过预告功能向玩家展示接下来将会出现的三个彩球颜色，让玩家能够提前规划策略。

游戏结束的条件是棋盘已满，没有任何空位可以放置新球。当程序准备生成新球但发现棋盘空位数量不足三个时，游戏应当结束，并显示最终得分。

1.8 图形界面要求

游戏界面必须使用中文制表符绘制边框，不能使用普通的英文字符如加号、减号、竖线等代替。作

业文档中提供了四种不同的中文制表符线型供选择，包括双线型、单线型、横双竖单型和横单竖双型，任选一种即可。程序需要实现两种界面模式：无分隔线模式和有分隔线模式。无分隔线模式下只绘制棋盘的外边框，内部格子之间没有分隔线。有分隔线模式下需要绘制完整的网格，包括外边框、内部横线、内部竖线以及横竖交叉点的十字连接符，总共需要使用十一种不同的中文制表符。

被选中的彩球需要用不同的符号进行高亮显示，例如用实心圆点或带圈的符号替代普通的空心圆符号。彩球移动时的动画效果需要沿BFS找到的最短路径逐格移动，每移动一格后刷新显示并加入适当的延时，形成视觉上的连续移动效果，且动画过程必须能够跨越棋盘内部的横线和竖线分隔符，不能出现瞬移。

2. 整体设计思路

本程序采用模块化设计，按照作业要求将代码划分为六个核心文件。头文件负责定义游戏数据结构体，封装棋盘数组、预告颜色、移动坐标、路径标记、统计信息和得分等所有游戏数据。主函数文件负责验证工具库版本正确性、设置控制台窗口大小和字体、显示主菜单循环，并处理每个菜单项结束后等待用户输入End的操作。菜单文件负责显示菜单选项、读取用户选择，并根据选择的菜单项调用相应的输入处理函数，同时负责接收用户输入的行列数值并验证其范围在7到9之间。

图形界面文件负责所有伪图形相关的绘制和交互功能，包括使用中文制表符绘制带分隔线或不带分隔线的游戏框架、处理鼠标事件的读取和坐标转换、实现彩球移动的逐格动画效果、以及实时更新右侧信息面板显示得分、预告球和颜色统计。工具文件实现了所有与游戏逻辑相关的核心函数，包括随机生成彩球、BFS路径搜索、消除检测与计分、空位统计等功能。

程序的数据结构设计了一个结构体，其中包含一个足够大的二维数组用于存储棋盘状态，0表示空格，1到7表示七种不同的彩球颜色。另有一个数组用于存储预告的三个彩球颜色，一个二维数组用于存储移动的起点和终点坐标，一个字符数组用于标记BFS寻找到的路径，还有两个统计数组分别用于存储当前棋盘各颜色球的数量与空位数、以及各颜色球累计消除的个数。结构体中还包含了棋盘的行数和列数以及当前得分。

整体程序流程为：用户从主菜单选择A到G七个选项之一。选择A到C时进入内部数组版本，在普通控制台界面中演示基础功能。选择D到G时进入图形界面版本，在伪图形界面中展示完整的游戏框架和交互效果。其中菜单D和E仅演示框架绘制和球的显示，菜单F演示鼠标单次移动操作，菜单G则实现了完整的游戏循环，包括移动、消除、生成新球、游戏结束判断等全部功能。

3. 主要功能的实现

3.1 游戏数据结构的封装

程序定义了一个名为Gamein的结构体，用于封装游戏运行所需的所有数据。该结构体包含一个20行20列的二维整型数组作为棋盘存储空间，由于棋盘最大为9×9，预留20的大小足以避免数组越界。棋盘数组中用0表示空格，用1至7表示七种不同颜色的彩球。结构体中还包含一个长度为4的整型数组用于存储预告的三个彩球颜色，一个2行2列的整型数组用于存储每次移动的起点坐标和终点坐标，一个字符型二维数组用于标记BFS寻找到的路径。此外，结构体还包含两个长度为9的整型统计数组，其中第一个统计数组用于存放当前棋盘上各颜色球的数量以及空位数量，第二个统计数组用于存放各颜色球自游戏开

始以来的累计消除个数。结构体中还记录了棋盘的行数和列数以及当前得分。将所有数据封装在同一个结构体中，便于在函数间传递，同时也避免了使用全局变量。

3.2 BFS路径搜索算法

路径搜索是游戏的核心功能之一。当用户选定起点和终点后，程序调用mover函数执行广度优先搜索。搜索开始前，程序首先初始化访问标记数组和前驱记录数组，然后将起点坐标加入队列并标记为已访问。搜索采用循环结构，每次从队列头部取出一个格子作为当前格子，然后依次检查该格子的上、下、左、右四个相邻格子。对于每个相邻格子，程序首先判断其是否在棋盘边界内，然后判断是否已经被访问过，接着判断是否为终点，最后判断是否为空格。如果是终点，程序记录前驱信息并设置找到标记后退出搜索循环。如果是空格，程序将其标记为已访问、记录前驱信息并将该格子加入队列尾部，等待后续探索。

搜索完成后，如果找到了路径，程序从终点开始沿着前驱记录逆向回溯到起点。在回溯过程中，程序将路径经过的所有格子（包括起点和终点）在路径标记数组中用星号字符标记出来。这些星号标记后续会被动画移动函数使用，指导彩球沿正确路径逐格移动。如果搜索未能找到路径，程序返回零表示移动不可行，调用方会给出相应的错误提示。

3.3 消除检测与得分计算

移动完成后，程序调用aftermove函数执行消除检测和得分计算。该函数首先执行球的移动，即交换起点和目标位置的球值。然后对每个连续段进行计时，程序还检查当前位置是否为该连续段的起点，具体方法是查看反方向相邻位置是否也是同色球，如果是则跳过计分，这确保了每个连续段只被计分一次，而双五连交叉点由于从不同方向检测时起点不同，仍能被重复计分，符合作业要求。计分公式采用消除数量减一乘以消除数量减二，例如消除五个球得分为四乘以三等于十二分，消除六个球得分为五乘以四等于二十分。

程序还统计了本次消除中各种颜色球的数量，并累加到结构体中的消除统计数组里。这些统计数据会被右侧信息面板读取并显示，让玩家了解每种颜色球累计消除了多少个。最后，程序将所有标记为待消除的位置清零，并将本次移动获得的总分累加到结构体的得分成员中。函数返回一表示发生了消除，返回零表示没有发生消除，调用方根据返回值决定是否生成新球。

3.4 随机生成彩球

程序中的get_array函数负责在棋盘上随机生成彩球。该函数接收生成个数、是否为预告模式、是否使用预告颜色等参数。生成时采用循环方式，每次随机生成一个行号和列号，如果该位置已经有球则重新生成，直到找到空位为止。球的颜色确定分为几种情况：在预告模式下，函数只生成预告球的颜色而不修改棋盘；在使用预告颜色的模式下，新球的颜色从之前生成的预告数组中按顺序取出；在普通模式下，颜色由随机数生成器从1至7中随机产生。需要特别注意的是，生成新球前程序会调用空位统计函数获取当前空位数量，如果空位少于需要生成的数量，则只生成实际存在的空位数。随机数生成器在程序开始时使用当前时间作为种子初始化一次，避免连续调用时产生相同的随机序列。

3.5 中文制表符框架绘制

图形界面的框架绘制由draw_first函数完成。该函数接收棋盘数据、是否绘制分隔线标志以及绘制起始坐标等参数。绘制过程分为三个主要步骤：绘制上边框、绘制中间行、绘制下边框。上边框的绘制从左到右进行，首先在起始位置输出左上角符号，然后根据棋盘列数输出相应数量的横线符号，如果有分隔线模式，还需要在每个列分隔位置输出向下连接符，最后在右上角输出右上角符号。中间行的绘

制采用双重循环，外层循环遍历每一行格子。每行先输出左边框竖线，然后内层循环遍历每一列，在有球的位置输出带颜色的彩球符号，在空格位置输出两个空格，如果是有分隔线模式，每列之间还需要输出竖线分隔符。每行格子输出完成后，如果不是最后一行且有分隔线模式，程序还需要输出一行由左连接符、横线、十字连接符、右连接符组成的水平分隔线。下边框的绘制与上边框对称，最后输出左下角符号、横线、向上连接符和右下角符号。

3.6 鼠标交互与坐标转换

鼠标交互功能由`xyin_mouse_array`函数实现。进入函数后，程序首先启用鼠标支持并隐藏光标，然后在循环中不断调用鼠标读取函数获取鼠标事件。当检测到鼠标移动事件时，程序根据鼠标的屏幕坐标计算出对应的棋盘行列位置，计算方法是：列坐标减去棋盘起始列偏移后除以每个格子占用的宽度，行坐标减去棋盘起始行偏移后除以每个格子占用的高度。如果计算出的行列在有效范围内，程序在提示栏显示当前光标位置，格式为行字母加列数字，例如第一行第一列显示为A1。当检测到鼠标左键单击事件时，程序根据当前光标位置进行判断：如果点击位置有球且当前没有选中的球，则记录该球的位置并用高亮符号重新显示该球，表示已选中；如果已有选中的球且点击位置为空，则记录目标位置并调用路径搜索函数，找到路径后调用动画移动函数执行移动。当检测到鼠标右键单击事件时，程序退出鼠标循环并返回上级菜单。

3.7 动画移动实现

动画移动功能由`draw_move`函数实现。该函数接收球当前的屏幕坐标作为参数。移动过程中，程序根据之前BFS搜索时标记的路径信息，逐步确定下一步移动的方向。由于路径标记数组中的星号记录了从起点到终点的完整路径，程序通过检查当前格子四个方向相邻格子的路径标记，即可判断下一步应该向上、下、左、右哪个方向移动。确定方向后，程序采用逐帧刷新的方式实现动画效果：先清除当前位置的彩球符号，然后在相邻位置绘制彩球符号，接着调用延时函数暂停一小段时间，最后更新当前坐标为新的位置。这个过程重复执行，直到彩球到达终点。动画过程中特别注意了对分隔线的处理，当球需要跨越横线或竖线时，程序会临时清除线上的符号，待球通过后再恢复，使得动画效果更加自然流畅。

3.8 右侧信息面板统计

右侧信息面板的绘制由`draw_menu`函数完成。函数首先调用统计函数更新当前棋盘的数据统计。统计函数遍历整个棋盘，对空格和各颜色球进行计数，结果存入结构体的`count`数组中，其中索引1至7存放七种颜色的数量，索引8存放空格数量。然后程序调用`draw_first`函数分别绘制得分区域、预告球区域和统计区域三个子框架。得分区域显示当前累计得分，预告球区域以迷你三格框架的形式显示接下来即将出现的三个彩球颜色，每个预告球用对应的颜色符号表示。统计区域以列表形式显示七种颜色各自的统计信息，包括当前棋盘上该颜色球的数量占棋盘总格子数的百分比，以及该颜色球自游戏开始以来的累计消除个数。统计信息每轮循环都会刷新，确保玩家能够实时了解游戏状态。

3.9 完整游戏循环

菜单G实现了完整的游戏循环。循环开始前，程序首先初始化棋盘，随机生成五个初始彩球，并生成一组预告球。进入主循环后，程序依次执行以下操作：绘制当前棋盘界面、绘制右侧信息面板、检查游戏是否结束、等待用户鼠标操作、执行移动操作、执行消除检测和得分计算、根据是否有消除决定是否生成新球、更新预告球信息。游戏结束的条件是棋盘空位不足三个且本次移动没有产生消除，此时程序显示游戏结束信息并退出循环。整个循环持续进行，直到玩家右键退出或游戏自然结束。这种循环结构保证了游戏的连续性和流畅性，玩家可以连续进行多次移动，直到无法继续操作为止。

4. 调试过程碰到的问题

一、得分计算错误。在开发菜单C的过程中，当连续五个同色球消除时，程序输出的得分是60分而不是预期的12分。经过分析发现，程序在遍历棋盘检测连续球时，对连续段中的每一个球都进行了一次计分。五个球每个都检测到连续五个同色球，各自累加了12分，导致总分变成60分。解决方法是修改计分逻辑，在每次检测到连续段时先判断当前位置是否为该连续段的起点，检查反方向上相邻位置是否也是同色球，如果是起点才进行计分，否则跳过。这样每个连续段只计分一次，而双五连交叉点由于从不同方向检测时起点不同，仍能重复计分，符合作业要求。

二、鼠标移动后需要按回车才能继续下一次操作。在菜单G完整游戏版本中，每次移动球后程序都会等待用户按下回车键才能进行下一次鼠标操作，严重影响了游戏体验。分析发现两个原因：一是主函数中每个菜单项结束后都要求用户输入End才能继续，菜单G作为持续运行的游戏不应受此限制；二是之前使用cin输入遗留的换行符会干扰鼠标事件的读取。解决方法是在主函数中单独处理菜单G，使其结束后直接继续菜单循环而不等待End输入，同时在进入图形界面彻底清空输入缓冲区。

三、统计面板显示数据错误。右侧统计面板中空位数量和颜色数量显示混乱。原因是统计函数中数组索引的含义不清晰，原本设计索引8存放空位数量，但实现时误写成了索引1自增，导致颜色1的数量包含了空位数。同时颜色索引转换也存在偏移错误，图形界面中颜色值因加7偏移而变成了8到14，还原时没有正确处理。解决方法是在头文件中明确定义数组索引的含义，重写统计函数，区分内部数组模式和图形界面模式，对颜色值进行正确的偏移转换，并将空位数量存放在正确的索引位置。

四、彩球移动动画无法跨越分隔线。在实现菜单F和菜单G的动画移动功能时，彩球沿路径移动的过程中会被棋盘内部的横线和竖线分隔符遮挡，无法形成连贯的跨越分隔线效果。原因是程序在移动球时只更新了球所在格子的显示内容，而没有考虑路径上分隔线的显示状态。解决方法是在动画循环中采用逐帧刷新策略，每移动一步时先清除当前位置的球符号，然后绘制当前位置的分隔线符号，接着在下一位置绘制球符号，最后加入短暂延时。这样球在移动过程中每次只前进一个格子，路径上的分隔线在球通过之前正常显示、通过之后恢复显示，视觉上形成了球跨越分隔线的效果，符合作业要求。

五、游戏结束时程序未能正确退出循环。在菜单C和菜单G中，当棋盘空位不足三个且本次移动没有产生消除时，游戏应当结束。但程序有时会继续运行，出现反复提示移动或无法正常退出循环的情况。原因是游戏结束的判断条件不够完整，只检查了空位数量是否为零，忽略了空位数量不足三个且本轮无消除时无法生成新球的情况。解决方法是在每次移动和消除完成后，先检查本次是否产生了消除，如果无消除则统计当前空位数量，若空位数量少于三个则判定游戏结束并退出循环，否则正常生成新球继续游戏。这样确保了游戏结束条件与作业规则完全一致。

5. 心得体会

5.1 心得体会

通过本次彩球游戏的开发，我在多个方面获得了宝贵的实践经验。首先，模块化编程的重要性给我留下了深刻印象，将程序拆分为六个文件并合理划分各模块的职责，使得代码结构清晰、便于调试和维护，核心工具函数被多个菜单共用，避免了大量重复代码。其次，BFS算法在实际问题中的应用让我对课

堂理论知识有了更直观的理解，从路径搜索到动画移动的实现过程中，我学会了如何记录前驱节点回溯路径、如何处理边界条件以及如何在保证正确性的前提下兼顾动画效果。在调试过程中，我遇到了得分重复计算、鼠标响应异常等问题，通过分析程序逻辑、输出中间变量等方式逐步定位并解决了这些bug，这不仅提升了我的调试能力，也让我认识到细节处理的重要性。最后，本次作业让我体会到合理规划开发顺序的必要性，从内部数组版本到图形界面版本、从基础功能到完整游戏，循序渐进地推进项目是保证最终顺利完成的有效策略。

5.2 代码重用与模块化设计分析

在完成本次彩球游戏作业的过程中，我深刻体会到前后小题之间的关联关系以及代码重用设计的重要性。

本次作业共包含七个菜单项，从菜单A到菜单G难度逐步递增，功能层层叠加。菜单A实现最基本的随机生成和数组打印，菜单B增加了路径搜索功能，菜单C将前两者结合形成完整的内部数组版游戏。菜单D和E在图形界面上绘制框架，菜单F增加了鼠标交互和动画移动，菜单G则整合所有功能形成完整图形版游戏。这种递进式的题目设计本身就在引导我们思考代码复用的策略。

在实现过程中，我尽可能让后面小题复用前面小题已完成的代码。例如，随机生成彩球的函数被菜单A、B、C、F、G共同使用；BFS路径搜索函数在菜单B、C、F、G中都被调用；消除检测和计分逻辑在菜单C和G中完全复用；中文制表符绘制函数同时服务菜单D、E、F、G四个图形界面版本。通过将通用功能抽取为独立的函数，我避免了大量重复代码的编写，也使得程序结构更加清晰。

6. 附件：源程序

```

/*****
*****
函数名称: moveroad
功    能: 路径搜寻
输入参数: Gamein 结构体 arrayin
返 回 值: int
说    明: 有路返回 1，没路返回 0
*****
*****/
int mover(Gamein& arrayin) {
    for (int i = 0; i < 10; i++)
        for (int j = 0; j < 10; j++)
            arrayin.moveroad[i][j] = '0';
    int startRow = arrayin.movexy[0][0];
    int startCol = arrayin.movexy[0][1];
    int endRow = arrayin.movexy[1][0];
    int endCol = arrayin.movexy[1][1];
    int visited[10][10] = { 0 };
    int prevRow[10][10];
    int prevCol[10][10];

    for (int i = 1; i <= arrayin.row; i++) {

int newRow = curRow + dr[dir];
        int newCol = curCol + dc[dir];

        if (newRow < 1 || newRow > arrayin.row ||
            newCol < 1 || newCol > arrayin.col) {
            continue;
        }
        if (visited[newRow][newCol]) {
            continue;
        }
        if (newRow == endRow && newCol == endCol)

for (int j = 1; j <= arrayin.col; j++) {
        prevRow[i][j] = -1;
        prevCol[i][j] = -1;
    }
    int queueRow[100], queueCol[100];
    int front = 0, rear = 0;

    queueRow[rear] = startRow;
    queueCol[rear] = startCol;
    rear++;
    visited[startRow][startCol] = 1;

    int dr[] = { -1, 1, 0, 0 };
    int dc[] = { 0, 0, -1, 1 };
    bool found = false;

    while (front < rear && !found) {
        int curRow = queueRow[front];
        int curCol = queueCol[front];
        front++;

        for (int dir = 0; dir < 4; dir++) {

            prevRow[newRow][newCol] = curRow;
            prevCol[newRow][newCol] = curCol;
            found = true;
            break;
        }
        if (arrayin.grid[newRow][newCol] == 0) {
            visited[newRow][newCol] = 1;
            prevRow[newRow][newCol] = curRow;
            prevCol[newRow][newCol] = curCol;
            queueRow[rear] = newRow;

```

```

        queueCol[rear] = newCol;
        rear++;
    }
}

if (found) {
    arrayin.moveroad[startRow][startCol] = '*';
    arrayin.moveroad[endRow][endCol] = '*';
    int r = endRow, c = endCol;
    while (!(r == startRow && c == startCol)) {
        int nextR = prevRow[r][c];
        int nextC = prevCol[r][c];
        if (!(nextR == startRow && nextC == startCol)) {
            arrayin.moveroad[nextR][nextC] = '*';
        }
        r = nextR;
        c = nextC;
    }
    return 1;
}
else {
    return 0;
}
}
/*****
*****
函数名称: aftermove
功    能: 内部数组移动及消除判断,算分
输入参数: Gamein 结构体 arrayin, score
返 回 值: int
说    明: 如果可以移动, 交换两点坐标并判断是否消除,没有
返回 0, 消除返回 1
*****
*****/
int aftermove(Gamein& arrayin) {
    int tmp;
    tmp
    arrayin.grid[arrayin.movexy[0][0]][arrayin.movexy[0][1]];
    arrayin.grid[arrayin.movexy[0][0]][arrayin.movexy[0][1]] =
    arrayin.grid[arrayin.movexy[1][0]][arrayin.movexy[1][1]];
    arrayin.grid[arrayin.movexy[1][0]][arrayin.movexy[1][1]] =
    tmp;
    int totalScore = 0;
    int toEliminate[10][10] = { 0 };
    int scored[10][10] = { 0 };

    for (int r = 1; r <= arrayin.row; r++) {
        for (int c = 1; c <= arrayin.col; c++) {
            int color = arrayin.grid[r][c];
            if (color == 0) continue;

            int dr[] = { 0, 1, 1, 1 };
            int dc[] = { 1, 0, 1, -1 };

            for (int dir = 0; dir < 4; dir++) {
                int count = 1;
                for (int step = 1; step <= 5; step++) {
                    int nr = r + dr[dir] * step;
                    int nc = c + dc[dir] * step;
                    if (nr < 1 || nr > arrayin.row || nc < 1
|| nc > arrayin.col) break;

                    if (arrayin.grid[nr][nc] == color)
                        count++;

                    else break;
                }
                for (int step = 1; step <= 5; step++) {
                    int nr = r - dr[dir] * step;
                    int nc = c - dc[dir] * step;
                    if (nr < 1 || nr > arrayin.row || nc < 1
|| nc > arrayin.col) break;

```

```

                    if (arrayin.grid[nr][nc] == color)
                        count++;

                    else break;
                }
            }

            if (count >= 5) {
                int prevR = r - dr[dir];
                int prevC = c - dc[dir];
                if (prevR >= 1 && prevR <=
                    prevC >= 1 && prevC <=
                        arrayin.grid[prevR][prevC]
                            continue;
            }

            int addScore = (count - 1) * (count -
2);

            totalScore += addScore;
            toEliminate[r][c] = 1;
            for (int step = 1; step <= 5; step++) {
                int nr = r + dr[dir] * step;
                int nc = c + dc[dir] * step;
                if (nr < 1 || nr > arrayin.row ||
nc < 1 || nc > arrayin.col) break;

                if (arrayin.grid[nr][nc] ==
color) toEliminate[nr][nc] = 1;

                else break;
            }
            for (int step = 1; step <= 5; step++) {
                int nr = r - dr[dir] * step;
                int nc = c - dc[dir] * step;
                if (nr < 1 || nr > arrayin.row ||
nc < 1 || nc > arrayin.col) break;

                if (arrayin.grid[nr][nc] ==
color) toEliminate[nr][nc] = 1;

                else break;
            }
        }
    }

    if (totalScore > 0) {
        int eliminateByColor[8] = { 0 };
        for (int r = 1; r <= arrayin.row; r++) {
            for (int c = 1; c <= arrayin.col; c++) {
                if (toEliminate[r][c]) {
                    int color = arrayin.grid[r][c];
                    int actualColor = color > 7 ? color -
7 : color;

                    if (actualColor >= 1 && actualColor
<= 7) {
                        eliminateByColor[actualColor]++;
                    }
                }
            }
        }
        for (int i = 1; i <= 7; i++) {
            arrayin.over[i+1] += eliminateByColor[i];
        }
        for (int r = 1; r <= arrayin.row; r++) {
            for (int c = 1; c <= arrayin.col; c++) {
                if (toEliminate[r][c]) {
                    arrayin.grid[r][c] = 0;
                }
            }
        }
        arrayin.score += totalScore;
        return 1;
    }
}

```



```

    }
    return 0;
}
/*****
*****
函数名称: count
功    能: 计数
输入参数: 内部数组 arrayin
返 回 值: void
说    明: 计数
*****
*****/
void count(Gamein& arrayin) {
    for (int i = 1; i <= 8; i++) {
        arrayin.count[i] = 0;
    }
    for (int r = 1; r <= arrayin.row; r++) {
        for (int c = 1; c <= arrayin.col; c++) {
            int color = arrayin.grid[r][c];
            if (color == 0) {
                arrayin.count[1]++;
            }
            else if (color >= 8 && color <= 14) {
                arrayin.count[color - 6]++;
            }
        }
    }
}
/*****
*****
函数名称: drawfirst
功    能: 绘制初始边框
输入参数: 内部数组 arrayin, 框线选项
返 回 值: void
说    明: 0 无框线, 1 有框线
*****
*****/
void draw_first(Gamein& arrayin, bool midline, int start_x, int start_y)
{
    int x, y;
    cct_showstr(start_x, start_y, " ┌ ", COLOR_HWHITE,
COLOR_BLACK, 1);
    cct_showstr(start_x+2, start_y, " — ", COLOR_HWHITE,
COLOR_BLACK, arrayin.col+ (arrayin.col - 1) * midline);
    if (midline) {
        for (int j = 1; j <= arrayin.col; j++)
        {
            cct_showstr(start_x + 2 + 2 * (j - 1) + 2 *
midline + 2, start_y, " ┴ ", COLOR_HWHITE, COLOR_BLACK, 1);
        }
        cct_showstr(start_x + arrayin.col * 2 + 2 + (arrayin.col - 1) * 2
* midline, start_y, " ┐ ", COLOR_HWHITE, COLOR_BLACK, 1);
        x = start_x;
        y = start_y + 1;
        for (int i = 1; i <= arrayin.row; i++) {
            cct_showstr(x, y, " │ ", COLOR_HWHITE,
COLOR_BLACK, 1);
            for (int j = 1; j <= arrayin.col; j++)
            {
                if (arrayin.grid[i][j] != 0) {
                    cct_showstr(start_x + 2 + 2 * (j - 1) + 2 *
1) * midline, y, " ○ ", arrayin.grid[i][j], COLOR_HWHITE, 1);
                }
                else {
                    cct_showstr(start_x + 2 + 2 * (j - 1) + 2 *
- 1) * midline, y, "   ", COLOR_HWHITE, COLOR_BLACK, 1);
                }
            }
            if (midline)
                cct_showstr(start_x + 2 + 2 * (j - 1) + 2 *

```

```

- 1) * midline + 2, y, " │ ", COLOR_HWHITE, COLOR_BLACK, 1);
        }
        cct_showstr(start_x + arrayin.col * 2 + 2 + (arrayin.col - 1)
* 2 * midline, y, " ┌ ", COLOR_HWHITE, COLOR_BLACK, 1);
        y++;
        if (midline && i < arrayin.row)
        {
            for (int j = 1; j <= arrayin.col; j++)
            {
                cct_showstr(x, y, " ┴ ",
COLOR_HWHITE, COLOR_BLACK, 1);
                cct_showstr(start_x + 2 + 2 * (j - 1)
+ 2 * (j - 1) * midline, y, " — ", COLOR_HWHITE, COLOR_BLACK,
1);
                cct_showstr(start_x + 2 + 2 * (j - 1)
+ 2 * (j - 1) * midline + 2, y, " ┴ ", COLOR_HWHITE,
COLOR_BLACK, 1);
                cct_showstr(start_x + arrayin.col * 2
+ 2 + (arrayin.col - 1) * 2 * midline, y, " ┐ ", COLOR_HWHITE,
COLOR_BLACK, 1);
            }
            y += midline;
        }
        cct_showstr(x, y, " ┌ ", COLOR_HWHITE, COLOR_BLACK,
1);
        cct_showstr(x + 2, y, " — ", COLOR_HWHITE,
COLOR_BLACK, arrayin.col + (arrayin.col - 1) * midline);
        if (midline) {
            for (int j = 1; j <= arrayin.col; j++)
            {
                cct_showstr(x + 2 + 2 * (j - 1) + 2 * (j - 1) * midline
+ 2, y, " ┴ ", COLOR_HWHITE, COLOR_BLACK, 1);
            }
        }
        cct_showstr(x + arrayin.col * 2 + 2 + (arrayin.col - 1) * 2 *
midline, y, " ┐ ", COLOR_HWHITE, COLOR_BLACK, 1);
        cct_setcolor();
        cout << endl;
    }
}
/*****
*****
函数名称: xyin_mouse_array
功    能: 读取鼠标
输入参数: 内部数组 arrayin
返 回 值: void
说    明:
*****
*****/
int xyin_mouse_array(Gamein& arrayin) {
    cct_enable_mouse();
    cct_setcursor(CCT_CURSOR_INVISIBLE);
    int MX, MY, MAction, keycode1, keycode2;
    int mx, my;
    int x = 0;
    int y = 2 + arrayin.row * 2;
    int score = 0;
    int selectedRow = 0, selectedCol = 0;
    bool selected = false;
    while (1) {
        bool Flag = false;
        int event = cct_read_keyboard_and_mouse(MX, MY,
MAction, keycode1, keycode2);
        if (event == CCT_MOUSE_EVENT) {
            int col = (MX - 2) / 2;
            int row = MY - 1;
            if (col % 2 == 0 && col / 2 < arrayin.col && row %
2 != 0 && row / 2 < arrayin.row)
            {
                char a = row / 2 + 'A';

```

```

}
    }
}
}
}

/*****
*****/

函数名称: draw_move
功    能: 实现彩球移动
输入参数: 内部数组 arrayin
返 回 值: void
说    明:

*****/

void draw_move(Gamein& arrayin,int mx,int my){
    int srcx = arrayin.movexy[0][0];
    int srey = arrayin.movexy[0][1];
    int dstx = arrayin.movexy[1][0];
    int dsty = arrayin.movexy[1][1];
    int way=0;
    int set[] = { 1,-1 };
    arrayin.moveroad[srcx][srey] = '0';
    while(1)
    {
        int i = 0;
        for (; i < 2; i++) {
            if (arrayin.moveroad[srcx + set[i]][srey] == '*')
            {
                arrayin.moveroad[srcx + set[i]][srey] = '0';
                way = 1;
                srcx += set[i];
                break;
            }
            if (arrayin.moveroad[srcx][srey + set[i]] == '*')
            {
                arrayin.moveroad[srcx][srey + set[i]] = '0';
                way = 3;
                srey += set[i];
                break;
            }
        }
        way += i;
        if (way == 1) {//下
            for (int j = 1; j <= 2; j++)
            {
                if (j % 2)
                    cct_showstr(mx, my, " ",
COLOR_HWHITE, COLOR_BLACK, 1);
                else
                    cct_showstr(mx, my, " — ",
COLOR_HWHITE, COLOR_BLACK, 1);
                cct_showstr(mx, my + 1, " ◎ ",
arrayin.grid[arrayin.movexy[0][0]][arrayin.movexy[0][1]],
COLOR_HWHITE, 1);
                cct_setcolor();
                my += 1;
                Sleep(100);
            }
        }
        if (way == 2) {//上
            for (int j = 1; j <= 2; j++)
            {
                if (j % 2)
                    cct_showstr(mx, my, " ",
COLOR_HWHITE, COLOR_BLACK, 1);
                else
                    cct_showstr(mx, my, " — ",
COLOR_HWHITE, COLOR_BLACK, 1);
                cct_showstr(mx , my-1, " ◎ ",
arrayin.grid[arrayin.movexy[0][0]][arrayin.movexy[0][1]].

```

```

draw_first(next,1,40,5);
Gamein.blank;
blank.col = 12;
blank.row = 8;
blank.grid[1][1] = 15;
for (int i = 8; i <= 14; i++) {
    blank.grid[i - 6][1] = i;
}
draw_first(blank, 0, 40, 9);
for (int i = 1; i <= 8; i++) {
    cct_showstr(x + 4, y + 8+i, ":", COLOR_HWHITE,
COLOR_BLACK, 1);
    cout << setfill('0') << setw(2) << arrayin.count[i];

    cout<< "/" <<setfill(' ') << setw(5) << fixed <<
setprecision(2) << arrayin.count[i] * 1.0 / (arrayin.col *
arrayin.row)*100 << "%" 消除-" << arrayin.over[i];
}
    cct_setcolor();
    cct_gotoxy(0, 22);
}
/*****
*****
函数名称: get_array
功    能: 内部数组生成
输入参数: Gamein& arrayin,int num,bool is_next,bool gotonext
返 回 值: void
说    明: num 是生成个数, is_next 是否是预告（预告了就不能生成）, gotonext 是生成预告
*****
*****/
void get_array(Gamein& arrayin, int num, bool is_next, bool gotonext)
{
    int cnt = 1, color;
    if (is_next) {
        for (int i = 1; i <= num; i++)
        {
            color = rand() % 7 + 1;
            arrayin.nextcolor[i] = color;
        }
    }
    else
    {
        while (cnt <= num) {
            int row = rand() % arrayin.row + 1;
            int col = rand() % arrayin.col + 1;
            if (gotonext)
                color = arrayin.nextcolor[cnt];
            else
                color = rand() % 7 + 1;
            if (arrayin.grid[row][col] != 0)
                continue;
            else {
                cnt++;
                arrayin.grid[row][col] = color;
            }
        }
    }
}
/*****
*****
函数名称: out_array
功    能: 内部数组输出
输入参数: Gamein 结构体 arrayin
返 回 值: void
说    明:
*****
*****/
void out_array(Gamein& arrayin, bool is_grid, bool is_road) {
    int firstn = 1:

```

```

*****
***** /

```

```
arrayin.col() &&
    (((arrayin.grid[xy[0] - 64][xy[1] - 48] != 0)
&& i == 0)
        || ((arrayin.grid[xy[0] - 64][xy[1] - 48]
== 0) && i == 1)))
    {
        arrayin.movexy[i][0] = xy[0] - 64;
        arrayin.movexy[i][1] = xy[1] - 48;
        cout << "输入为" << xy[0] << "行" << xy[1]
<< "列"
            << endl;
        i++;
        break;
    }
else if (cin.good() && (((arrayin.grid[xy[0] -
64][xy[1] - 48] == 0) && i == 0)
        || ((arrayin.grid[xy[0] - 64][xy[1] - 48] != 0)
&& i == 1))) {
    cout << cinerror[i] << endl;
    if (cin.fail())
    {
        cin.clear();

        cin.ignore((numeric_limits<streamsize>::max)(), '\n');
    }
    cout << cinxxy[i];
    cct_getxy(x, y);
}
else {
    cout << "输入错误，请重新输入";
    if (cin.fail())
    {
        cin.clear();

        cin.ignore((numeric_limits<streamsize>::max)(), '\n');
    }
    cct_gotoxy(x, y);
    cout << " ";
    cct_gotoxy(x, y);
}
}
```